

Liability Conditions Standard (LCOS)

Liability Conditions Standard (LCOS)

Architecture: LIGA™ — Liability Governance Architecture
Category: Governance & Enforcement
Subcategory: Liability Governance
Type: Parent Standard
Governed Space: Liability Conditions
Standard ID: LIGA-LCOS-001
Origin ID: OOF.PS.LIGA.05.00.01
Version: 1.0
Status: Canonical · Open Standard
Effective Date: 22 September 2026
Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem
AI-Readable: Yes
Authority: OOF® — OriginOpen® Foundation
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Canonical Definition

The Liability Conditions Standard (LCOS) governs how the conditions relevant to potential Liability under an applicable Governed Liability Basis are identified, structured, classified, represented, evaluated as governed condition states, and maintained so that subsequent Liability governance can determine whether the required conditional structure exists for Liability-Relevant Connection and later Liability Determination.

A Liability Condition is a governed requirement, circumstance, state, threshold, dependency, exception, exclusion, or other conditional element whose existence, absence, satisfaction, failure, change, or applicability may materially affect whether potential Liability can proceed under a Governed Liability Basis.

LCOS determines the governed state of Liability Conditions.

It does not determine Liability.

Liability Condition ≠ Liability.

Operational Role

The operational role of LCOS is:

to transform an applicable Governed Liability Basis and Governed Liability Relationship Structure into an explicit, traceable, and evaluable Liability Condition Structure.

LBS establishes:

On what governed basis can Liability potentially exist?

LRS establishes:

Who or what participates, and through what relationships?

LCOS establishes:

What must be true, false, present, absent, satisfied, failed, changed, or excepted for potential Liability to proceed under that Basis?

Core Question

What conditions must exist, fail, change, or be satisfied for potential Liability to proceed toward determination under the applicable Liability Basis?

Position in the LIGA™ Governance Flow

The canonical LIGA™ Governance Flow is:

Liability Context → Liability Object → Liability Basis → Liability Relationships → Liability Conditions → Liability Connection → Liability Determination → Liability Scope & Allocation → Liability State & Change → Liability Transfer & Recourse → Liability Resolution & Discharge → Liability Continuity & Finality LCOS governs:

Liability Conditions

It receives:

Governed Liability Relationship Structure

together with the relevant Governed Liability Object and Governed Liability Basis.

It produces:

Governed Liability Condition Structure

for:

Liability Connection Governance

Canonical Transition

LCOS governs the transition:

Governed Liability Relationship Structure → Condition Identification → Condition Requirement Structuring → Condition State Governance → Exception & Exclusion Governance → Governed Liability Condition Structure The transition establishes whether the conditional structure required by the applicable Liability Basis has been sufficiently governed for subsequent Connection analysis.

Governed Space

LCOS governs:

Liability Condition Identification, Condition Origin, Condition Identity, Condition Type, Condition Requirement, Condition Applicability, Condition State, Condition Satisfaction, Condition Failure, Condition Absence, Condition Presence, Positive Conditions, Negative Conditions, Threshold Conditions, Dependent Conditions, Alternative Conditions, Concurrent Conditions, Sequential Conditions, Cumulative Conditions, Conditional Conditions, Temporal Conditions, Subject-Specific Conditions, Relationship-Specific Conditions, Object-Specific Conditions, Basis-Specific Conditions, Condition Exceptions, Condition Exclusions, Condition Waivers where applicable, Condition Dependencies, Condition Conflict, Condition Uncertainty, Condition Change, Condition Reassessment, Condition Continuity, and Condition Governance Sufficiency.

Liability Condition

A Liability Condition is a governed conditional element whose state may materially affect whether potential Liability can proceed under an applicable Liability Basis.

A Condition may concern:

a Subject, a Relationship, the Liability Object, the Liability Basis, conduct, authority, control, state, timing, knowledge, performance, non-performance, threshold, exception, or another materially relevant governed fact or state.

The existence of a Condition does not establish Liability.

Basis Before Conditions

Liability Conditions do not exist methodologically in isolation.

They must be governed relative to a relevant Liability Basis.

Therefore:

Liability Basis → Liability Conditions not:

Observed Condition → Invented Liability Basis The Basis explains why the Condition matters.

The Condition explains what must be evaluated under that Basis.

Liability Basis and Liability Conditions

The architecture preserves the fundamental distinction:

Liability Basis $\neq$ Liability Conditions.

A Liability Basis establishes the governed foundation under which Liability may potentially exist.

Liability Conditions establish what must be true, false, present, absent, satisfied, failed, or otherwise applicable under that Basis.

Basis Existence and Condition Satisfaction

The existence of a valid or applicable Liability Basis does not mean its Conditions are satisfied.

Therefore:

Applicable Liability Basis $\neq$ Satisfied Liability Conditions.

A Basis may be fully established while one or more required Conditions remain unsatisfied.

Condition Identification

Every materially relevant Condition should be explicitly identified.

Condition identification should establish:

what the Condition is, why it is relevant, which Basis gives it relevance, which Subject, Relationship, Object, or state it concerns, and how its state can be represented.

Condition Identity

A materially distinct Condition should possess a stable governed identity.

Condition Identity should remain distinguishable from:

Condition State, Condition Evidence, Condition Satisfaction, and Condition Outcome.

Therefore:

Condition Identity $\neq$ Condition State.

Condition Origin

Condition Origin identifies the governed source from which a Condition becomes relevant.

A Condition may originate from:

the Liability Basis, a governed rule, an obligation, a relationship, an authority structure, an operational state, a defined threshold, or another applicable source.

Therefore:

Condition Origin $\neq$ Condition Satisfaction.

Condition Requirement

A Condition Requirement expresses what must be evaluated.

Examples may include whether:

a particular state existed, a threshold was crossed, an obligation applied, an authority existed, an action occurred, an action failed to occur, a relationship was active, a temporal requirement was met, or an exception applied.

The requirement must be explicit enough to support a governed Condition State.

Condition Requirement and Outcome

The requirement itself is not its outcome.

Therefore:

Condition Requirement $\neq$ Condition Satisfaction.

A system must distinguish:

what was required, from:

whether that requirement was satisfied.

Condition Applicability

Not every Condition associated with a Liability Basis necessarily applies to every Liability Object, Subject, Relationship, or lifecycle state.

LCOS must determine Condition Applicability before interpreting Condition State.

Therefore:

Condition Existence $\neq$ Condition Applicability.

Applicable Condition

An Applicable Condition is a Condition determined to be relevant to the specific governed Liability inquiry.

Applicability may depend upon:

Object characteristics, Subject role, Relationship type, Basis scope, temporal state, jurisdiction-specific external input where applicable, or another governed factor.

Applicability does not determine satisfaction.

Non-Applicable Condition

A Condition may exist within the wider Basis structure but be non-applicable to the governed matter.

Non-applicability should be explicit.

Therefore:

Non-Applicable Condition $\neq$ Failed Condition.

The Condition does not fail merely because it does not apply.

Condition Type

Conditions may be classified methodologically according to their governance function.

Potential types may include:

Positive Condition, Negative Condition, Threshold Condition, Temporal Condition, Dependency Condition, Sequential Condition, Alternative Condition, Concurrent Condition, Cumulative Condition, Subject-Specific Condition, Relationship-Specific Condition, Object-Specific Condition, Basis-Specific Condition, Exception Condition, or another governed type.

Classification supports evaluation.

It does not determine the Condition State.

Positive Condition

A Positive Condition requires the presence, existence, occurrence, or satisfaction of a defined state.

Example structure:

Required State must exist.

Therefore:

Positive Condition Satisfaction requires governed support for the required presence.

Negative Condition

A Negative Condition requires the absence or non-occurrence of a defined state.

Example structure:

Disqualifying State must not exist.

Negative Conditions must not be evaluated using the same logic as Positive Conditions without preserving the difference.

Absence and Negative Conditions

Absence may itself be a required Condition State.

However:

Absence of Evidence $\neq$ Evidence of Required Absence by default.

The architecture must distinguish unknown state from established absence.

Threshold Condition

A Threshold Condition requires a governed value, state, or measure to cross, reach, remain below, or otherwise satisfy a defined threshold.

Threshold governance should preserve:

the threshold, the measured or governed value, the comparison rule, and the resulting state.

Threshold Crossing

Crossing a threshold may change a Condition State.

It does not automatically establish Liability.

Therefore:

Threshold Satisfaction $\neq$ Liability Determination.

Temporal Condition

A Temporal Condition depends upon time.

It may concern:

when something occurred, how long a state existed, whether an action occurred within a required period, whether a Relationship was active at the relevant time, or whether a requirement had already expired.

Therefore:

Current Condition State $\neq$ Historical Condition State by default.

Subject-Specific Condition

A Condition may apply to one Liability Subject but not another.

Therefore:

Condition Satisfaction for Subject A $\neq$ Condition Satisfaction for Subject B.

Subject-specific states must remain separately governed.

Relationship-Specific Condition

A Condition may apply only within a specific Liability Relationship.

The same Subject may therefore have different Condition States across different Relationships.

Therefore:

One Subject $\neq$ One Condition State across all Relationships.

Object-Specific Condition

A Condition may apply only to a particular Governed Liability Object or Object component.

Therefore:

Condition Satisfaction for Object A $\neq$ Condition Satisfaction for Object B.

Object boundaries must remain preserved.

Basis-Specific Condition

A Condition may be required under one Liability Basis but irrelevant under another.

Therefore:

Condition under Basis A $\neq$ Condition under Basis B by default.

Multiple Bases require Basis-specific Condition governance.

Multiple Conditions

A single Liability Basis may contain multiple relevant Conditions.

These should remain separately identifiable.

Therefore:

Multiple Liability Conditions $\neq$ One Composite Condition by default.

Concurrent Conditions

Several Conditions may need to be satisfied simultaneously.

Example:

Condition A + Condition B + Condition C

The architecture should preserve the requirement for concurrence.

Therefore:

Individual Condition Satisfaction $\neq$ Concurrent Condition Satisfaction.

Alternative Conditions

A Basis may allow alternative conditional paths.

Example:

Condition A OR Condition B

LCOS should preserve the alternative structure rather than incorrectly requiring both.

Therefore:

Alternative Conditions $\neq$ Concurrent Conditions.

Cumulative Conditions

Several Conditions may accumulate to produce the required conditional state.

Cumulative governance should preserve:

each individual Condition, its state, and the cumulative rule.

Therefore:

Cumulative Condition State $\neq$ Individual Condition State.

Sequential Conditions

Some Conditions must occur or be evaluated in a defined sequence.

Example:

Condition A $\rightarrow$ Condition B $\rightarrow$ Condition C Sequence should remain explicit.

Therefore:

Condition Order $\neq$ Condition Identity.

Dependent Conditions

A Condition may depend upon another Condition.

Example:

Condition B applies only if Condition A is satisfied.

Therefore:

Condition Dependency $\neq$ Condition Satisfaction.

Dependency must be evaluated separately.

Conditional Conditions

A Condition may become applicable only when another governed state exists.

This creates conditional applicability.

Example:

If State X exists $\rightarrow$ evaluate Condition Y.

Therefore:

Trigger State $\neq$ Condition Y Satisfaction.

Nested Conditions

A complex Basis may contain Conditions within Conditions.

Nested structures should remain explicit rather than being flattened into an ambiguous single requirement.

Therefore:

Nested Condition Structure $\neq$ Single Condition.

Condition Network

Complex matters may contain interconnected Conditions.

A Condition Network may include:

dependencies, alternatives, sequences, thresholds, exceptions, and Subject-specific branches.

The network should remain reconstructable.

Therefore:

Condition Network $\neq$ Liability Relationship Network.

The two are different governed structures.

Condition State

A Condition State represents the governed status of an identified Condition at a defined lifecycle moment.

Possible states may include:

Satisfied, Not Satisfied, Partially Satisfied, Failed, Present, Absent, Not Applicable, Unknown, Uncertain, Disputed, Pending, or another governed state.

Satisfied Condition

A Condition is Satisfied when sufficient governed input supports the required Condition State under the applicable evaluation logic.

Satisfaction applies to the Condition.

It does not establish Liability.

Therefore:

Satisfied Liability Condition $\neq$ Established Liability.

Unsatisfied Condition

A Condition may be identified and applicable while remaining unsatisfied.

Therefore:

Applicable Condition $\neq$ Satisfied Condition.

An unsatisfied required Condition may prevent potential Liability from progressing under the affected Basis or conditional path.

Failed Condition

A Failed Condition represents a governed state in which the required Condition has not been met under the applicable evaluation structure.

However:

Failed Condition $\neq$ Absence of Liability under every possible Basis.

Another Liability Basis or conditional path may remain relevant.

Partially Satisfied Condition

A Condition may be only partially satisfied where the requirement contains divisible elements or the governed state remains incomplete.

Partial satisfaction should not be silently treated as full satisfaction.

Therefore:

Partial Satisfaction $\neq$ Full Satisfaction.

Unknown Condition State

The architecture should support an explicit:

Unknown

state where the Condition cannot yet be reliably evaluated.

Therefore:

Unknown Condition State $\neq$ Failed Condition.

and:

Unknown Condition State $\neq$ Satisfied Condition.

Uncertain Condition State

A Condition State may be uncertain where available governed inputs support more than one plausible interpretation.

Uncertainty should remain explicit.

Therefore:

Condition Uncertainty $\neq$ Authority to Manufacture Condition Certainty.

Disputed Condition State

Different parties, systems, or governed sources may support conflicting Condition States.

The dispute should remain represented.

Therefore:

Disputed Condition $\neq$ Automatically Failed Condition.

Pending Condition

A Condition may be applicable but not yet evaluable because required information, events, or dependencies remain outstanding.

Therefore:

Pending Condition ≠ Unsatisfied Condition by default.

Condition Evidence

Evidence may support Condition evaluation.

ADIT® may govern the evidence itself.

LCOS governs the Condition State derived for Liability purposes.

Therefore:

Evidence of a Condition ≠ Condition State by itself.

Validated Input

VALIDOS™ may validate Condition-relevant information.

Validation may increase confidence in the input.

However:

Validated Input ≠ Satisfied Liability Condition by default.

LCOS must still apply the Condition Requirement.

Condition and Operational Reality

ORA™ may provide actual states relevant to Condition evaluation.

Examples may include:

who operated a system, whether control existed, whether an action occurred, or whether a required operational state existed.

Operational Reality is an input.

Therefore:

Operational State ≠ Liability Condition State by itself.

Condition and Accountability

AGA™ may provide Responsibility, Accountability, Authority, or Attribution states relevant to a Condition.

However:

Accountability State ≠ Liability Condition Satisfaction by default.

The applicable Liability Basis determines whether and how the state matters.

Condition and Harm

HAGA™ may provide governed Harm states where Harm is relevant to a Condition.

However:

Established Harm ≠ Satisfied Liability Conditions by default.

Liability may require additional or different Conditions.

Harm Absence

Some Liability Bases may not require Harm.

Therefore:

Absence of Harm ≠ Absence of Liability Conditions.

LIGA™ must not assume that Harm is universally required.

Condition and Conduct

Conduct may be relevant to a Condition.

The architecture should distinguish:

the conduct itself, the Condition requiring or evaluating the conduct, and the resulting Condition State.

Therefore:

Conduct $\neq$ Liability Condition.

Condition and Breach

A Breach may become relevant to a Liability Condition under an applicable Basis.

However:

Breach $\neq$ Liability Condition Satisfaction by default.

and:

Breach $\neq$ Liability.

The governing Basis determines its significance.

Condition and Fault

Fault may be relevant under some Liability Bases and irrelevant under others.

Therefore:

Fault $\neq$ Universal Liability Condition.

LCOS must not impose fault where the applicable Basis does not require it.

Condition and Responsibility

Responsibility may be relevant to some Conditions.

However:

Responsibility $\neq$ Liability Condition Satisfaction by default.

Responsibility remains separately governed by AGA™.

Condition and Authority

Authority may be required, prohibited, exceeded, absent, or otherwise relevant under a Condition.

However:

Authority State $\neq$ Liability by itself.

The Condition must be evaluated within the applicable Basis.

Condition and Control

Control may be a relevant Condition under some Bases.

However:

Control $\neq$ Liability.

and:

Control Relationship $\neq$ Satisfied Control Condition by default.

The required control state must be evaluated explicitly.

Condition and Knowledge

Knowledge or awareness may become relevant under some Liability Bases.

LCOS may govern whether the required knowledge-related Condition is:

satisfied, not satisfied, unknown, or disputed.

It must not assume knowledge merely from outcome or proximity.

Therefore:

Outcome Knowledge $\neq$ Prior Knowledge by default.

Condition and Capability

A Subject's capability to act may become relevant under some Liability Bases.

Capability should remain distinct from:

authority, obligation, actual action, and Liability.

Therefore:

Capability $\neq$ Duty $\neq$ Action $\neq$ Liability.

Condition and Obligation

An obligation may be relevant to a Condition.

However:

Obligation Existence $\neq$ Obligation Breach.

and:

Obligation Breach $\neq$ Liability Determination.

These stages must remain distinct.

Condition and Performance

Performance or non-performance may determine a Condition State under some Bases.

The architecture should preserve:

required performance, actual performance, evaluation, and Condition State.

Therefore:

Performance Record $\neq$ Condition State by itself.

Condition Exception

An Exception is a governed state capable of modifying, limiting, displacing, or preventing the ordinary operation of a Condition.

Exceptions should be explicitly identified.

Therefore:

Exception $\neq$ Condition Absence.

The Condition may exist while an Exception changes its effect.

Exception Applicability

An Exception must itself be evaluated for applicability.

Therefore:

Exception Existence $\neq$ Applicable Exception.

A generally available Exception may not apply to the specific matter.

Exception Satisfaction

An applicable Exception may require its own requirements to be satisfied.

Therefore:

Applicable Exception $\neq$ Satisfied Exception.

The architecture should preserve the distinction.

Condition Exclusion

An Exclusion removes a defined matter, Subject, Object, state, or circumstance from the operation of a Condition or conditional structure where applicable.

Therefore:

Exclusion $\neq$ Failed Condition.

Exclusion changes applicability.

Exception and Exclusion

Exceptions and Exclusions should remain distinct where their governance functions differ.

A methodological distinction may be:

Exception modifies ordinary application.

Exclusion removes defined applicability.

Therefore:

Exception $\neq$ Exclusion.

Waiver

Where relevant to the applicable Basis, a waiver may modify the operation of a Condition.

The existence and validity of a waiver must not be assumed.

Therefore:

Waiver Assertion $\neq$ Governed Waiver State.

LCOS does not determine jurisdiction-specific legal validity unless provided through an authoritative external input.

Condition Conflict

Two Conditions may conflict.

Example:

one governed source requires State A, while another applicable structure appears to require State not-A.

LCOS should preserve the conflict.

Therefore:

Condition Conflict $\neq$ Automatic Governance Failure.

The conflict must be explicit for downstream handling.

Competing Conditional Paths

Multiple Liability Bases or Basis interpretations may produce competing Condition Structures.

These should not be prematurely merged.

Therefore:

Competing Condition Structures $\neq$ One Unified Condition Structure by default.

Each materially distinct path should remain traceable.

Alternative Basis Conditions

Where multiple Liability Bases exist, each may possess a separate Condition Structure.

Failure under Basis A does not establish failure under Basis B.

Therefore:

Failed Conditions under Basis A $\neq$ Failed Conditions under Basis B.

Condition Dependency Across Bases

A Condition under one Basis may depend on a governed state associated with another Basis.

Such dependency should be represented without merging the Bases.

Therefore:

Cross-Basis Dependency $\neq$ Basis Identity.

Condition Evaluation

Condition evaluation determines the governed state of an identified Condition against its explicit Requirement.

The evaluation should preserve:

Condition Identity, Requirement, applicable input, evaluation logic, temporal state, uncertainty, and resulting Condition State.

Condition Evaluation and Legal Judgment

LCOS provides methodological governance of Condition evaluation.

It does not replace jurisdiction-specific courts, regulators, arbitrators, insurers, contractual authorities, or other legally empowered decision-makers.

Therefore:

Methodological Condition Evaluation $\neq$ Jurisdiction-Specific Legal Judgment.

Condition State at Time T

Condition evaluation must use the relevant temporal state.

A Condition satisfied today may not have been satisfied when the relevant occurrence happened.

Therefore:

Condition State at T2 $\neq$ Condition State at T1 by default.

Condition Change

A Condition State may change over time.

Example:

Pending $\rightarrow$ Satisfied or:

Satisfied $\rightarrow$ No Longer Satisfied or:

Unknown $\rightarrow$ Established The change should remain traceable.

Condition Change and Liability

A Condition change may affect potential or established Liability governance.

It does not automatically create or eliminate Liability.

Therefore:

Condition Change $\neq$ Liability Change by default.

The affected downstream governance must be reassessed where necessary.

Condition Correction

A previously recorded Condition State may be incorrect.

Correction should preserve:

the earlier state, the corrected state, the correction reason, and the relevant supporting input.

Therefore:

Condition Correction ≠ Historical Erasure.

Condition Reassessment

New information may require reassessment of:

Condition Applicability, Condition Requirement, Condition State, Exception State, Exclusion State, or dependencies.

Therefore:

Condition Reassessment ≠ Historical Rewrite.

Condition Re-Entry

Later lifecycle governance may reveal that an earlier Condition must be revisited.

Re-entry should occur at the earliest materially affected LCOS governance point.

Therefore:

Condition Re-Entry ≠ New Liability Origin by default.

Condition Continuity

Condition Identity and state history should remain traceable throughout the Liability lifecycle.

Therefore:

Condition Continuity ≠ Condition Immutability.

A Condition may change while its history remains continuous.

Current Condition State and History

The current Condition State must not erase earlier states.

Therefore:

Current Condition State ≠ Complete Condition History.

Historical states may remain relevant to later Determination, review, Transfer, Recourse, Resolution, or re-entry.

Condition Versioning

Material changes to Condition representation or evaluation may require versioning.

Therefore:

New Condition Version ≠ New Condition by default.

The persistent Condition Identity should remain distinguishable from its versions.

Condition Supersession

A Condition may be superseded by a later Condition or requirement.

Supersession should preserve the earlier Condition and its historical applicability.

Therefore:

Condition Supersession ≠ Historical Deletion.

Condition Termination

A Condition may cease to apply.

Termination should preserve:

when it ceased, why it ceased, and its prior states.

Therefore:

Condition Termination ≠ Condition Erasure.

Condition Network Change

A Condition Network may change as:

Conditions are added, removed, superseded, reclassified, or dependencies change.

The earlier network state should remain reconstructable.

Therefore:

Condition Network Change $\neq$ Historical Network Replacement.

Condition State and Liability-Relevant Connection

A sufficiently governed Condition Structure becomes an input to Liability Connection governance.

However:

Satisfied Conditions $\neq$ Liability-Relevant Connection.

LCNS determines whether the required Liability-specific connection exists.

Conditions Without Connection

All required Conditions may appear satisfied while the required Liability-Relevant Connection remains absent.

Therefore:

Condition Satisfaction $\neq$ Connection Satisfaction.

This is a critical architectural boundary.

Connection Without Satisfied Conditions

A strong factual or operational connection may exist while required Liability Conditions remain unsatisfied.

Therefore:

Connection $\neq$ Condition Satisfaction.

Neither substitutes for the other.

Conditions and Liability Attachment

Even where Conditions are satisfied, Liability has not yet attached.

Therefore:

Satisfied Liability Conditions $\neq$ Liability Attachment.

Attachment belongs to LDS.

Conditions and Liability Determination

LCOS does not determine Liability.

The governed sequence remains:

Basis $\rightarrow$ Relationships $\rightarrow$ Conditions $\rightarrow$ Connection $\rightarrow$ Determination not:

Condition Satisfaction $\rightarrow$ Automatic Liability

Conditions and Liability Scope

Condition governance does not establish how far Liability extends.

Therefore:

Condition Scope $\neq$ Liability Scope.

LSAS owns Liability Scope.

Conditions and Liability Allocation

Condition states may later influence Liability Determination or Allocation.

They do not allocate Liability themselves.

Therefore:

Condition Satisfaction $\neq$ Liability Allocation.

Conditions and Multiple Subjects

Different Subjects may have different Condition States within the same Liability matter.

Therefore:

Condition Satisfaction for One Subject $\neq$ Condition Satisfaction for All Subjects.

The architecture must preserve Subject-specific conditional structures.

Conditions and Multiple Relationships

A Subject may satisfy a Condition in one Relationship while not satisfying it in another.

Therefore:

Condition State in Relationship A $\neq$ Condition State in Relationship B.

Relationship-specific governance must remain explicit.

Conditions and Liability Networks

A complex Liability Relationship Network may contain multiple Condition Structures.

The architecture should not assume one Condition State applies across the entire network.

Therefore:

One Liability Network $\neq$ One Condition Structure by default.

No Outcome-to-Condition Shortcut

The existence of Harm, Loss, Damage, failure, or another adverse outcome must not be used to reverse-engineer Condition Satisfaction without governed evaluation.

Therefore:

Adverse Outcome $\neq$ Satisfied Liability Conditions.

No Relationship-to-Condition Shortcut

The existence of a Relationship does not establish the Conditions associated with that Relationship.

Therefore:

Liability Relationship $\neq$ Satisfied Liability Conditions.

No Basis-to-Condition Shortcut

The existence of a Basis does not establish the Conditions under that Basis.

Therefore:

Liability Basis $\neq$ Satisfied Liability Conditions.

No Evidence-to-Condition Shortcut

Evidence concerning a relevant fact does not automatically determine the Condition State.

The evidence must be evaluated against the explicit Condition Requirement.

Therefore:

Evidence $\neq$ Condition Satisfaction.

No Claim-to-Condition Shortcut

A Claim may assert that Conditions are satisfied.

The assertion does not establish their governed state.

Therefore:

Claim Assertion $\neq$ Condition Satisfaction.

Claims remain outside the LIGA™ governed domain as a separate governance object.

No Fault Assumption

LCOS must not treat fault as a universal requirement.

Some Liability Bases may require fault.

Others may not.

Therefore:

Fault-Based Condition Structure $\neq$ Universal Liability Condition Structure.

No Harm Assumption

LCOS must not treat Harm as a universal Condition.

Some Liability structures may concern obligations, statutory states, guarantees, restitution, or other matters not requiring governed Harm.

Therefore:

Harm $\neq$ Universal Liability Condition.

No Single-Condition Assumption

Liability governance should not assume that one observed state is sufficient merely because it appears significant.

Therefore:

Significant Condition $\neq$ Complete Condition Structure.

All materially required Conditions must remain governed.

No Condition Collapse

Distinct Conditions must not be merged merely because they concern similar facts.

For example:

authority, control, knowledge, obligation, and capability may interact while remaining separate Conditions.

Therefore:

Related Conditions $\neq$ Identical Conditions.

Conditional Structure Completeness

A governed Condition Structure should represent, where applicable:

required Conditions, alternative Conditions, dependent Conditions, sequential Conditions, exceptions, exclusions, temporal requirements, Subject-specific requirements, Relationship-specific requirements, and unresolved Condition States.

Completeness is governed relative to the applicable Basis.

Condition Governance Sufficiency

Condition governance is sufficient when subsequent Liability Connection governance can reliably determine:

which Conditions apply, what each Condition requires, which Subjects or Relationships each Condition concerns, what temporal state applies, which Conditions are satisfied, which are not satisfied, which remain unknown or disputed, which Exceptions or Exclusions apply, which dependencies exist, and what uncertainty remains.

Sufficiency does not require Liability Determination.

Principal Output

The principal output of LCOS is:

Governed Liability Condition Structure

The output should preserve:

Condition identities, Basis references, Subject references, Relationship references, Object references, Condition Requirements, Condition Types, Applicability States, Condition States, dependencies, sequences, alternatives, thresholds, Exceptions, Exclusions, temporal states, uncertainty, conflicts, change history, and continuity information.

Governance Requirements

A conforming LCOS implementation must:

identify materially relevant Liability Conditions;

associate each Condition with an applicable Governed Liability Basis;

preserve Condition Identity independently of Condition State;

establish explicit Condition Requirements;

determine Condition Applicability before interpreting satisfaction;

distinguish Positive and Negative Conditions where applicable;

preserve threshold and temporal Conditions;

preserve Subject-specific and Relationship-specific Conditions;

preserve multiple, concurrent, alternative, cumulative, sequential, and dependent Conditions;

explicitly represent unknown, uncertain, disputed, partial, and pending Condition States;

govern Exceptions and Exclusions separately from ordinary Condition failure;

preserve Basis-specific conditional structures;

prevent Harm, Loss, Damage, Claim, Fault, Responsibility, Relationship, or adverse outcome from automatically

establishing Condition Satisfaction;

preserve Condition change, correction, reassessment, and historical state;

prevent Condition Satisfaction from being treated as Liability-Relevant Connection;

prevent Condition Satisfaction from being treated as Liability Attachment;

prevent Condition Satisfaction from being treated as Liability Determination;

preserve material uncertainty for downstream governance;

support recursive re-entry where later information affects earlier Condition governance; and

produce a Governed Liability Condition Structure sufficient for Liability Connection governance.

Governance Logic

The canonical LCOS governance logic is:

Governed Liability Relationship Structure + Governed Liability Basis + Governed Liability Object ↓

Identify Potential Liability Conditions

↓

Establish Condition Identity

↓

Establish Condition Origin

↓

Establish Condition Requirement

↓

Determine Condition Applicability

↓

Classify Condition Type

↓

Associate Subject / Relationship / Object / Basis

↓

Establish Temporal State

↓

Identify Condition Dependencies

↓

Identify Alternative / Concurrent / Sequential / Cumulative Structures

↓

Evaluate Condition State

↓

Evaluate Exceptions and Exclusions

↓

Preserve Unknown / Uncertain / Disputed / Pending States

↓

Resolve or Preserve Condition Conflicts

↓

Record Condition Change / Correction / Reassessment

↓

Preserve Condition Continuity

↓

Assess Condition Governance Sufficiency

↓

Governed Liability Condition Structure

↓

Handoff to Liability Connection Governance

Governance Outputs

LCOS may produce:

Liability Condition Identity

Liability Condition Requirement

Liability Condition Type

Condition Applicability State

Positive Condition State

Negative Condition State

Threshold Condition State

Temporal Condition State

Subject-Specific Condition State

Relationship-Specific Condition State

Object-Specific Condition State

Basis-Specific Condition State

Concurrent Condition Structure

Alternative Condition Structure

Cumulative Condition Structure

Sequential Condition Structure

Dependent Condition Structure

Nested Condition Structure

Liability Condition Network

Satisfied Condition State

Unsatisfied Condition State

Failed Condition State

Partially Satisfied Condition State

Unknown Condition State

Uncertain Condition State

Disputed Condition State

Pending Condition State

Condition Exception State

Condition Exclusion State

Condition Conflict Record

Condition Change Record

Condition Correction Record

Condition Reassessment Record

Condition Continuity Record

Condition Governance Sufficiency State

The principal output is:

Governed Liability Condition Structure

Critical Distinctions

LCOS preserves the following distinctions:

Liability Condition ≠ Liability Liability Basis ≠ Liability Conditions Applicable Liability Basis ≠ Satisfied Liability
Conditions Condition Identity ≠ Condition State Condition Origin ≠ Condition Satisfaction Condition Requirement ≠
Condition Satisfaction Condition Existence ≠ Condition Applicability Non-Applicable Condition ≠ Failed Condition
Positive Condition ≠ Negative Condition Absence of Evidence ≠ Evidence of Required Absence by default Threshold
Satisfaction ≠ Liability Determination Current Condition State ≠ Historical Condition State by default Condition
Satisfaction for Subject A ≠ Condition Satisfaction for Subject B One Subject ≠ One Condition State across all
Relationships Condition Satisfaction for Object A ≠ Condition Satisfaction for Object B Condition under Basis A ≠
Condition under Basis B by default Multiple Liability Conditions ≠ One Composite Condition by default Individual
Condition Satisfaction ≠ Concurrent Condition Satisfaction Alternative Conditions ≠ Concurrent Conditions Cumulative
Condition State ≠ Individual Condition State Condition Order ≠ Condition Identity Condition Dependency ≠ Condition
Satisfaction Trigger State ≠ Dependent Condition Satisfaction Nested Condition Structure ≠ Single Condition Condition
Network ≠ Liability Relationship Network Satisfied Liability Condition ≠ Established Liability Applicable Condition ≠
Satisfied Condition Failed Condition ≠ Absence of Liability under every possible Basis Partial Satisfaction ≠ Full
Satisfaction Unknown Condition State ≠ Failed Condition Unknown Condition State ≠ Satisfied Condition Condition
Uncertainty ≠ Authority to Manufacture Condition Certainty Pending Condition ≠ Unsatisfied Condition by default
Evidence of a Condition ≠ Condition State by itself Validated Input ≠ Satisfied Liability Condition by default Operational
State ≠ Liability Condition State by itself Accountability State ≠ Liability Condition Satisfaction by default Established
Harm ≠ Satisfied Liability Conditions by default Absence of Harm ≠ Absence of Liability Conditions Conduct ≠ Liability
Condition Breach ≠ Liability Condition Satisfaction by default Breach ≠ Liability Fault ≠ Universal Liability Condition
Responsibility ≠ Liability Condition Satisfaction by default Authority State ≠ Liability by itself Control ≠ Liability Outcome
Knowledge ≠ Prior Knowledge by default Capability ≠ Duty ≠ Action ≠ Liability Obligation Existence ≠ Obligation
Breach Obligation Breach ≠ Liability Determination Performance Record ≠ Condition State by itself Exception ≠
Condition Absence Exception Existence ≠ Applicable Exception Applicable Exception ≠ Satisfied Exception Exclusion
≠ Failed Condition Exception ≠ Exclusion Waiver Assertion ≠ Governed Waiver State Condition Conflict ≠ Automatic
Governance Failure Competing Condition Structures ≠ One Unified Condition Structure by default Failed Conditions

under Basis A ≠ Failed Conditions under Basis B Cross-Basis Dependency ≠ Basis Identity Methodological Condition Evaluation ≠ Jurisdiction-Specific Legal Judgment Condition State at T2 ≠ Condition State at T1 by default Condition Change ≠ Liability Change by default Condition Correction ≠ Historical Erasure Condition Reassessment ≠ Historical Rewrite Condition Re-Entry ≠ New Liability Origin by default Condition Continuity ≠ Condition Immutability Current Condition State ≠ Complete Condition History New Condition Version ≠ New Condition by default Condition Supersession ≠ Historical Deletion Condition Termination ≠ Condition Erasure Condition Network Change ≠ Historical Network Replacement Satisfied Conditions ≠ Liability-Relevant Connection Condition Satisfaction ≠ Connection Satisfaction Connection ≠ Condition Satisfaction Satisfied Liability Conditions ≠ Liability Attachment Condition Satisfaction ≠ Automatic Liability Condition Scope ≠ Liability Scope Condition Satisfaction ≠ Liability Allocation Condition Satisfaction for One Subject ≠ Condition Satisfaction for All Subjects Condition State in Relationship A ≠ Condition State in Relationship B One Liability Network ≠ One Condition Structure by default Adverse Outcome ≠ Satisfied Liability Conditions Liability Relationship ≠ Satisfied Liability Conditions Liability Basis ≠ Satisfied Liability Conditions Evidence ≠ Condition Satisfaction Claim Assertion ≠ Condition Satisfaction Fault-Based Condition Structure ≠ Universal Liability Condition Structure Harm ≠ Universal Liability Condition Significant Condition ≠ Complete Condition Structure Related Conditions ≠ Identical Conditions

Cross-Architecture Boundary

LCOS may receive governed states from other OOF® architectures where those states become relevant to an applicable Liability Condition.

GOA™ may provide Authority, Scope, Delegation, and governance states.

ORA™ may provide Operational Reality and historical operational states.

AGA™ may provide Responsibility, Accountability, Authority, and Attribution states.

HAGA™ may provide Harm states where Harm is conditionally relevant.

ADIT® may provide evidence supporting Condition evaluation.

VALIDOS™ may validate Condition-relevant inputs.

OBIDENTITY® may provide Identity, Ownership, Authority, Permission, Provenance, Transfer, and Continuity states.

AIG® may provide AI behavioral and governance states.

ASGA™ may provide autonomous-system Authority, Permission, Delegation, Override, Execution, and other governed states.

PGA™ may provide Prediction-related states where a Liability Condition depends upon prediction, reliance, or predictive governance.

VFM® may provide Value-related states where those states become conditionally relevant.

The canonical interoperability principle is:

An externally governed state may satisfy, fail, modify, or otherwise inform a Liability Condition without becoming Liability and without transferring governance ownership of its original domain to LIGA™.

Canonical Boundary

LCOS begins when a:

Governed Liability Relationship Structure

exists together with a sufficiently established Governed Liability Basis and Governed Liability Object to identify the Conditions relevant to potential Liability.

LCOS governs:

Condition Identification, Condition Identity, Condition Origin, Condition Requirement, Condition Applicability, Condition Type, Condition State, Condition Satisfaction, Condition Failure, Positive and Negative Conditions, Thresholds, Temporal Conditions, Subject-Specific Conditions, Relationship-Specific Conditions, Object-Specific Conditions, Basis-Specific Conditions, Concurrent Conditions, Alternative Conditions, Cumulative Conditions, Sequential Conditions, Dependent Conditions, Nested Conditions, Exceptions, Exclusions, Condition Conflict, Condition Uncertainty, Condition Change, Condition Correction, Condition Reassessment, Condition Re-Entry, Condition Continuity, and Condition Governance Sufficiency.

LCOS does not determine:

Liability-Relevant Connection, Liability Attachment, Liability Determination, Liability Scope, Liability Allocation, Liability State, Liability Transfer, Recourse, Liability Resolution, Discharge, or Finality.

LCOS ends when a:

Governed Liability Condition Structure

is sufficiently established for Liability Connection governance.

Complete Lifecycle Relationship

LCOS occupies a pre-determination stage within the complete LIGA™ lifecycle:

Potential Liability → Attached / Determined Liability → Active Liability Governance → Liability Change → Transfer / Recourse where applicable → Resolution / Discharge → Continuity / Re-entry where required → Finality Condition governance primarily operates before Liability Determination.

However, Condition States may later require reassessment through lifecycle re-entry.

Therefore:

Liability Determination does not make earlier Condition governance historically irrelevant.

Recursive Governance

Later lifecycle information may reveal that:

a Condition was missing, a Condition was incorrectly classified, a Condition State was wrong, an Exception applied, an Exclusion was overlooked, or the applicable temporal state was incorrect.

LCOS should re-enter the earliest materially affected governance point.

Re-entry must preserve the earlier state.

Therefore:

Reassessment changes the governed understanding. It does not erase governance history.

Historical Integrity

LCOS must preserve the distinction between:

what the Condition State actually was, what was known at the relevant time, what was represented at the relevant time, and what was discovered later.

Therefore:

Later Condition Knowledge ≠ Earlier Condition Knowledge.

Historical integrity is necessary for reconstructable Liability governance.

Condition Readiness

A Condition Structure is ready for handoff when the materially required Conditions are sufficiently: identified, applicable, structured, evaluated, bounded, temporally aligned, and traceable.

Uncertainty may remain where explicitly preserved.

Therefore:

Condition Readiness ≠ Liability Determination Readiness.

Liability Connection remains a separate governance stage.

Canonical Governance Principle

The governing principle of LCOS is:

A Liability Basis defines why Liability may potentially exist. Liability Conditions define what must be true under that Basis. Neither establishes Liability by itself.

Canonical Closing Statement

Potential Liability cannot proceed reliably from Basis and Relationships directly to Determination.

The conditions required by the applicable Basis must first be made explicit, evaluated in the correct Subject, Relationship, Object, and temporal context, and preserved without converting satisfaction, failure, uncertainty, Harm, Fault, Breach, or adverse outcome into an automatic Liability conclusion.

Conditions establish what must be true. They do not establish Liability.

Govern the Conditions before determining the Connection.

Nahllad

Liability Conditions Standard (LCOS)

Architecture:

LIGA™ — Liability Governance Architecture

Category:

Governance & Enforcement

Subcategory:

Liability Governance

Canonical Definition:

The Liability Conditions Standard (LCOS) governs how the conditions relevant to potential Liability under an applicable Governed Liability Basis are identified, structured, classified, represented, evaluated as governed condition states, and maintained so that subsequent Liability governance can determine whether the required conditional structure exists for Liability-Relevant Connection and later Liability Determination.

Governed Space:

Liability Conditions

LCOS establishes what must be true, false, present, absent, satisfied, failed, changed, or excepted for potential Liability to proceed under an applicable Liability Basis. It governs Condition Requirements, applicability, states, thresholds, dependencies, alternative and concurrent Conditions, temporal Conditions, Exceptions, Exclusions, uncertainty, and change without determining Liability itself.

Liability Basis defines why Liability may potentially exist. Liability Conditions define what must be true under that Basis. Neither establishes Liability by itself.

→ [View Standard About standard](#)

Internal Module Flow

LCNIM — Liability Condition Identification Module

LCNRM — Liability Condition Requirement Module

LCNSM — Liability Condition State Module

LCNXM — Liability Condition Exception Module

LCNCM — Liability Condition Continuity Module

Related Documents

[About Liability Conditions Standard \(LCOS\)](#)

[LIGA™ Architecture Map](#)

[LIGA™ Liability Governance Architecture — Liability Governance Layer](#)

[LIGA™ Complete Standards & Modules Index](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the MIP® Protection & Licensing terms or contact OOF® directly.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/content/liga/liga-lcos.html>

OOF® · MIP® Protected · Free permitted use · Commercial/derivative use may require authorization · Active Protection

Canonical source: <https://originopenfoundation.org/content/liga/liga-lcos.html>